

AWS Well-Architected Framework Review — Batch 2: Event Creation and Event Discovery/RSVP Flows

1 Executive Summary

This Batch 2 Well-Architected Review of ROCA's Event Creation and Event Discovery/RSVP flows identified 21 architectural gaps across all six pillars, with the most severe risks concentrated in payment processing security and RSVP data integrity. Critically, all five gaps from Batch 1 — missing Step Functions orchestration, absent DLQs, no WAF/GuardDuty, missing EventBridge, and unresolved JWT token rotation — compound significantly in Batch 2 due to the financial and concurrency-sensitive nature of these workflows.

The most urgent finding is that ROCA's Stripe integration has four critical PCI DSS v4.0 compliance gaps: API keys stored in plaintext Lambda environment variables rather than Secrets Manager [1, 2], no webhook signature verification via HMAC-SHA256 [3], unprotected payment endpoints lacking WAF rate-limiting [4], and absent CloudTrail audit logging [5]. Since PCI DSS v4.0 became fully mandatory in March 2025 [6], these gaps represent immediate regulatory and financial exposure. Equally critical, the RSVP capacity enforcement logic lacks database-level concurrency control, meaning concurrent requests for the last available seat can both confirm — a classic lost-update race condition requiring Redis atomic Lua scripts [7] and PostgreSQL SELECT FOR UPDATE locking [8]. Additionally, the rrule.js library used for recurring event generation has documented DST-related defects that shift event times after timezone transitions [9, 10], directly impacting weekly service schedules for congregations.

The proposed 90-day remediation roadmap totals 40–55 engineering days at approximately \$190–\$250/month in incremental infrastructure costs. Phase 1 (Days 1–30) should be executed immediately, prioritizing Secrets Manager migration, webhook verification, atomic capacity counters, and RDS Proxy deployment to close compliance and data integrity gaps before addressing Phase 2 orchestration and Phase 3 search performance optimizations.

2 Introduction and Batch 2 Scope

This report presents the second batch (of four) of the AWS Well-Architected Framework Review for the ROCA (Religious Organization Calendar Application) platform. Batch 2 evaluates two core user-facing workflows: the **Event Creation** flow and the **Event Discovery & RSVP** flow, assessing them against all six Well-Architected pillars — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability.

The Event Creation flow is a 6-step wizard guiding organizers through Basic Info, Schedule & Recurrence (with RRULE support), Location & Resources, Tickets & Payment (Stripe integration), Volunteers & Roles, and Review & Publish [11]. Conditional branching logic shows the RRULE builder when `isRecurring=true`, capacity settings when `registrationRequired=true`, payment configuration when `ticketPrice>0`, and the role builder when `needsVolunteers=true` [11]. The event state machine progresses through DRAFT → PUBLISHED → ACTIVE → COMPLETED → ARCHIVED, with CANCELLED branching from DRAFT, PUBLISHED, or ACTIVE states, and cancellation of paid events triggering REFUND_PENDING → REFUND_COMPLETE [11].

The Event Discovery & RSVP flow is a 5-step journey from Browse/Search through Event Detail, RSVP/Checkout, Confirmation, and the My Events Dashboard [11]. The RSVP state machine follows PENDING → CONFIRMED → CHECKED_IN → ATTENDED, with CANCELLED branching from PENDING or CONFIRMED, and a parallel WAITLISTED → PROMOTED → CONFIRMED path for capacity-constrained events [11]. Capacity enforcement logic routes RSVPs to CONFIRMED when `currentAttendees < capacity`, to WAITLISTED when waitlist is enabled, or to REJECTED when capacity is full [11].

The underlying architecture uses Node.js/Express on AWS Lambda behind API Gateway (REST), PostgreSQL 15 RDS Multi-AZ, Redis 7 (ElastiCache), S3/CloudFront for static assets, SES/SNS for notifications, Stripe for payment processing, WebSocket via Socket.io, and JWT/OAuth 2.0 authentication. Recurring events are stored in the `event_series` table with an RRULE string (RFC 5545, VARCHAR(500)), a JSONB event template, and a JSONB exceptions array [12, 13]. Payment processing uses `stripePaymentIntentId` with webhook-driven asynchronous status updates [13, 14].

This review cross-references five critical gaps identified in Batch 1 (New Member Onboarding): missing Step Functions orchestration, absent JWT token rotation, no dead-letter queues (DLQs), missing WAF/GuardDuty protections, and no EventBridge event bus. These gaps compound significantly in Batch 2's payment-processing and concurrency-sensitive workflows.

Chart Explanation: This chart summarizes the 21 architectural gaps identified across the six Well-Architected pillars, categorized by severity. Security leads with 4 critical and 1 high-severity gap driven by PCI DSS compliance risks. Reliability follows with 3 critical gaps related to RSVP race conditions and connection management. Operational Excellence has 1 critical gap in cancellation orchestration plus high-severity RRULE parsing risks.

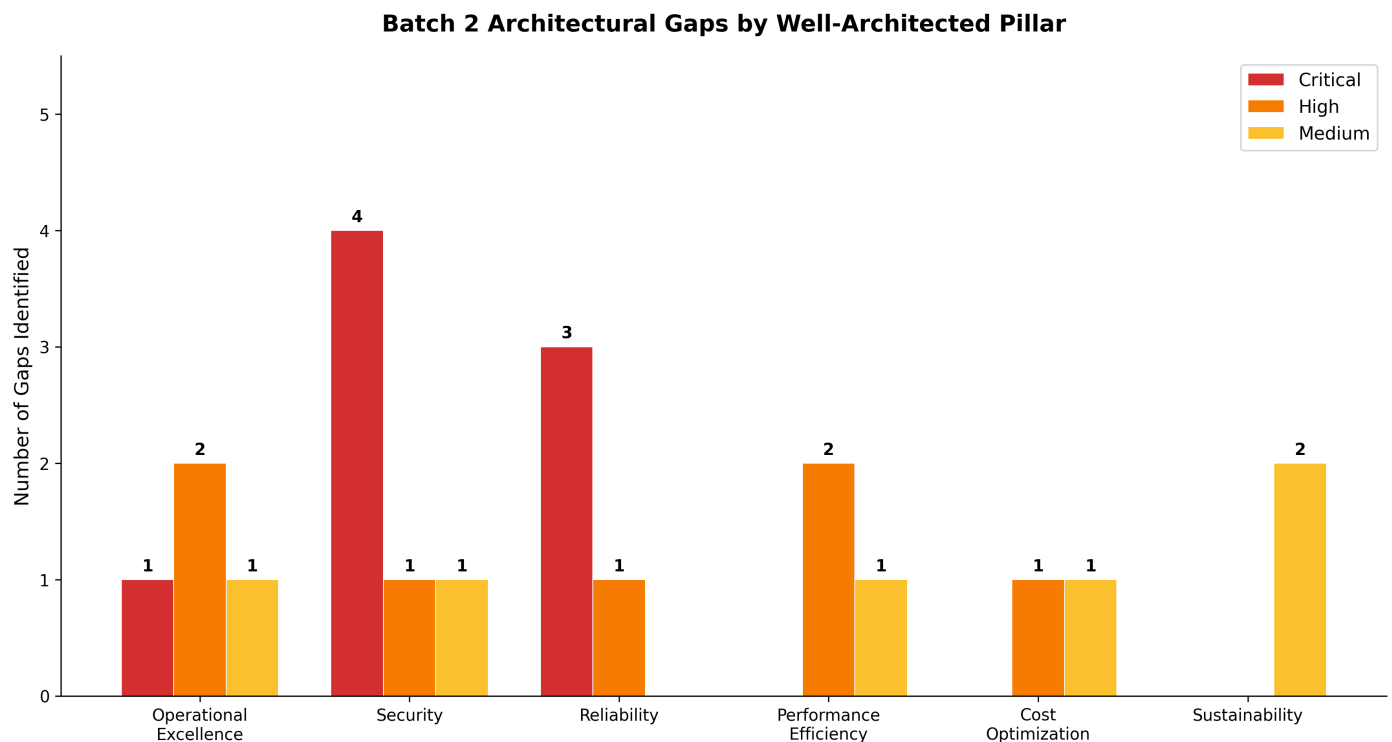


Figure 1: Batch 2 Architectural Gaps by Pillar

3 Batch 1 Gap Cross-Reference and Compounding Risks

The five critical gaps identified in Batch 1 do not merely persist into Batch 2 — they compound in severity due to the financial and concurrency-sensitive nature of the Event Creation and RSVP flows.

Missing Step Functions orchestration is the most impactful carryover. In Batch 1, the absence affected multi-step onboarding workflows. In Batch 2, it directly undermines the event cancellation cascade, which must atomically coordinate across four dependent flows: Discovery, Volunteering, Notifications, and Payments [11]. The cancellation workflow requires impact analysis (attendees, volunteers, payments), multi-channel notification dispatch (in-app at T+0, email at T+5min, SMS at T+15min, refund status at T+1hr), and tiered refund processing (full automatic refund if >48h before start, 75% partial if 24–48h, always full for organizer-initiated cancellations) [11]. Without Step Functions’ saga orchestration pattern, a failure at any step — particularly Stripe refund API calls — leaves the system in an inconsistent state with no compensating transaction mechanism [15, 16].

Absent DLQs pose heightened risk for Stripe webhook processing. Webhooks drive asynchronous payment status updates (pending → processing → completed → failed → refund_pending → refunded) [13], and a dropped webhook event means a payment confirmation or refund acknowledgment is silently lost. Similarly, RSVP state transitions triggered by capacity changes or waitlist promotions require guaranteed delivery that standard Lambda invocations without DLQs cannot provide.

Missing WAF and GuardDuty now expose payment-processing endpoints (POST /payments/process, POST /payments/{paymentId}/refund) to rate-limiting attacks, credential stuffing, and bot-driven abuse [13]. While Batch 1’s exposure was limited to authentication endpoints, Batch 2’s payment surface area introduces direct financial risk. The absence of **EventBridge** prevents fan-out of critical domain events such as event.cancelled, rsvp.confirmed, and payment.refunded across bounded contexts [17]. Finally, **JWT token rotation** gaps identified in Batch 1 remain unresolved, meaning compromised tokens could authorize fraudulent RSVPs or payment operations.

4 Pillar 1: Operational Excellence

RRULE Parsing Reliability and DST Edge Cases. The ROCA architecture stores recurrence rules as RFC 5545 RRULE strings in the event_series table [12]. The widely-used rrule.js library has multiple documented defects where recurrence series shift time-of-day after DST transitions — for example, a daily event at 12:00 in America/Denver jumps by one hour after the spring-forward boundary [9]. The library’s internal UTC representation causes DTSTART with TZID to produce incorrect local times [10], and UNTIL with TZID generates non-compliant strings

missing the trailing Z for UTC [18]. For a religious organization managing weekly services and recurring community events, these defects could cause services to appear at the wrong time for entire congregations. The recommended remediation is migration to `rrule-temporal`, the first fully RFC-5545-compliant JavaScript/TypeScript library built on the TC39 Temporal API with native timezone and non-Gregorian calendar support [19]. For ICS file interoperability, `node-ical` provides robust RRULE expansion with timezone-aware date handling, EXDATE, and RECURRENCE-ID support [20].

Recurring Event Instance Generation Architecture. The current design stores a master record with an RRULE string, but the architecture documentation does not specify whether instances are materialized (pre-generated into the database) or expanded virtually at read time [21]. Materialization simplifies queries and supports per-instance exceptions such as cancellations and rescheduling, but requires cascading updates when the series rule changes [22]. The recommended pattern is to use EventBridge Scheduler with cron expressions for timezone-aware recurring triggers that automatically adjust for DST [23], combined with Step Functions Map states to fan out instance generation across a rolling window — for example, materializing 90 days ahead with periodic refresh [24].

Cancellation Cascade Orchestration. Event cancellation affects four cross-flow dependencies without any orchestration mechanism [11]. The saga pattern using Step Functions would orchestrate compensating transactions where each step (cancel instances → revoke tickets → process refunds) has a corresponding compensation action that rolls back on failure [15]. Step Functions provides built-in error handling, retry with exponential backoff, and Catch states for routing to compensation Lambda functions [16]. EventBridge would propagate cancellation events across bounded contexts — a single event `.cancelled` event on the default bus can fan out to notification, refund, and analytics targets via rules [17].

Observability Gaps. The current architecture lacks structured logging for event state transitions (DRAFT → PUBLISHED → ACTIVE → COMPLETED) [11] and RSVP state transitions (PENDING → CONFIRMED → CHECKED_IN → ATTENDED) [11]. Without CloudWatch custom metrics and alarms on transition failure rates, payment processing errors, and RRULE parsing exceptions, operational teams have no visibility into degraded workflows. The volunteer auto-schedule engine, which scores assignments by skill match (40%), availability (30%), workload balance (20%), and preference (10%) [11], also lacks observability into scoring distribution and assignment failure rates.

5 Pillar 2: Security — PCI DSS Compliance and Payment Processing

PCI DSS v4.0 Compliance Gaps. PCI DSS v4.0 became fully mandatory (including previously future-dated requirements) as of March 31, 2025 [6]. While ROCA achieves scope reduction through client-side tokenization via Stripe.js — ensuring raw card numbers never reach API Gateway or Lambda, with only Stripe-generated payment tokens passed to the backend [25, 26] — several critical gaps remain in the surrounding infrastructure.

Stripe API Key Management. The architecture documentation references `STRIPE_WEBHOOK_SECRET` as a configuration value [14], but there is no evidence of AWS Secrets Manager usage for Stripe secret keys. Storing API keys in plaintext Lambda environment variables is a common serverless anti-pattern that violates PCI DSS key management requirements [27]. AWS Secrets Manager enables rotation, management, and retrieval of API keys throughout their lifecycle [1], and CloudTrail records all Secrets Manager API calls for audit purposes [2]. The remediation requires migrating Stripe secret keys and webhook signing secrets to Secrets Manager with automatic rotation, KMS customer-managed key encryption, and least-privilege IAM policies.

Webhook Signature Verification. Stripe documentation mandates securing webhook endpoints by verifying the `Stripe-Signature` header using `constructEvent()` with the raw request body, the signature header, and the endpoint signing secret — this uses HMAC-SHA256 to confirm authenticity and includes timestamp-based replay attack protection [3]. Without this verification, the payment Lambda function accepts forged webhook events, enabling attackers to manipulate payment statuses. Idempotency checks using the event ID are also required to prevent duplicate processing [28].

Payment Endpoint Protection and Audit Logging. AWS WAF should be attached to API Gateway with rate-limiting rules specifically on payment endpoints (`POST /payments/process`, `POST /payments/{paymentId}/refund`) and bot mitigation rules [13]. AWS Security Hub supports PCI DSS v4.0.1 compliance checks with automated controls [4], and AWS Config conformance packs can verify PCI DSS v4.0 operational best practices continuously [5]. Additional gaps include overly permissive IAM execution roles on payment Lambda functions and the risk of accidentally logging sensitive payment metadata to CloudWatch without PII masking [27]. All payment transactions require CloudTrail audit logging with tamper-evident log integrity validation.

The following table summarizes the PCI DSS compliance gap assessment:

Gap	Current State	Required State	Severity
Stripe API key storage	Lambda env vars [14]	Secrets Manager with KMS + rotation [1, 2]	Critical
Webhook signature verification	Not implemented	HMAC-SHA256 via <code>constructEvent()</code> [3]	Critical
WAF on payment endpoints	Not deployed	Rate-limiting + bot mitigation rules [4]	Critical
Payment audit logging	No CloudTrail integration	CloudTrail + Config conformance packs [2, 5]	Critical
IAM least-privilege (payment Lambdas)	Overly permissive [27]	Scoped to Stripe + Secrets Manager only	High
PII masking in Cloud-Watch	Not implemented	Log filter patterns for payment metadata [27]	Medium

Table: PCI DSS v4.0 Compliance Gap Assessment for ROCA Payment Flows

6 Pillar 3: Reliability — Race Conditions, Capacity Enforcement, and Connection Management

RSVP Race Conditions and Overbooking Risk. The RSVP capacity logic checks `currentAttendees < capacity` to determine whether to confirm or waitlist an RSVP [11], but there is no evidence of database-level concurrency control to prevent overbooking under concurrent requests. Without explicit locking, two simultaneous RSVPs for the last available seat could both read the same capacity count and both be confirmed — a classic lost-update anomaly [8]. The architecture requires a two-tier concurrency control strategy.

At the Redis layer, Lua scripts execute atomically on the server, enabling a check-and-decrement pattern that prevents overbooking at sub-millisecond latency [7]. This serves as a fast pre-check layer before the PostgreSQL write, reducing database contention during traffic spikes. The pattern is: if the Redis counter for `event:{id}:capacity` is greater than zero, atomically decrement and proceed; otherwise, route to waitlist logic immediately.

At the PostgreSQL layer, `SELECT ... FROM events WHERE id = \ $1 FOR UPDATE` within a transaction atomically reads then decrements capacity [8]. For higher throughput, `SELECT FOR UPDATE SKIP LOCKED` allows concurrent workers to skip already-locked rows instead of waiting, preventing serialization bottlenecks [29]. Optimistic locking with a version column provides an alternative that avoids holding locks during computation — the `UPDATE` includes `WHERE version = \ $old_version` and increments the version, with zero affected rows indicating a concurrent modification requiring retry.

Waitlist Promotion Ordering. When a cancellation frees a seat, the next `PENDING` waitlist entry must be promoted to `CONFIRMED` in strict FIFO order. SQS FIFO queues with message group IDs keyed by event ID ensure exactly-once, ordered processing of waitlist promotions [30]. The consumer Lambda atomically transitions the next `PENDING` entry using optimistic locking: `UPDATE reservations SET status = 'CONFIRMED', version = version + 1 WHERE id = \ $1 AND status = 'PENDING' AND version = \ $old_version` to prevent double-booking.

Lambda-to-PostgreSQL Connection Management. Lambda's ephemeral execution model creates unpredictable connection surges that can exhaust PostgreSQL connection limits. RDS Proxy pools and reuses database connections, handling unpredictable surges without exhausting connection limits and automatically connecting to a standby DB instance during failures [31]. This is essential for RSVP processing during high-traffic events where hundreds of concurrent Lambda invocations may attempt simultaneous database writes.

Distributed Locking for Critical Sections. For operations requiring mutual exclusion beyond database rows — such as serializing capacity checks per event — Redlock provides the canonical distributed lock algorithm for Redis, designed for environments where multiple processes must operate on shared resources in a mutually exclusive way [32]. On AWS ElastiCache, lock TTL should be set shorter than Lambda's timeout to prevent orphaned locks.

7 Pillar 4: Performance Efficiency — Search and Event Discovery

PostgreSQL Full-Text Search Optimization. The current architecture uses PostgreSQL tsvector on event title and description for full-text search [12], with the events endpoint supporting filters for startDate, endDate, type, status, and visibility with pagination [13]. PostgreSQL ships tsvector for indexing, websearch_to_tsquery for parsing user input, and ts_rank for relevance ordering — for most applications, this is exactly the job Elasticsearch or OpenSearch gets deployed to do [33]. However, the approach feels lower-level compared to dedicated search engines and lacks built-in faceted aggregations and advanced relevance tuning [34].

The immediate optimization is ensuring GIN indexes are created on tsvector columns combined with B-tree indexes on the most common filter columns (date, category, capacity, status). There is no evidence in the current architecture documentation that GIN indexes have been explicitly configured, which means full-text queries may be performing sequential scans.

Redis Cache-Aside Pattern. The recommended pattern is to hash query parameters to form a cache key, check Redis first, return cached data on a hit, or query the database on a miss and store the result with a TTL [35]. For event search results, 30–60 second TTLs balance freshness with performance; for popular or trending events, sorted sets with longer TTLs of 5–15 minutes reduce database load. Redis server-pushed invalidation messages can notify clients when tracked keys change [36], enabling near-real-time cache coherence without polling.

Scaling Breakpoints and OpenSearch Evaluation. PostgreSQL-first search is the most cost-effective approach for up to approximately 100K–500K events. OpenSearch Serverless automatically provisions and scales resources to provide millisecond query response times during changing demand [37], with a minimum entry of 0.5 OCU (half vCPU, 3 GB RAM, 60 GB storage) scaling in 1-OCU increments [38]. The tradeoff is data sync complexity, typically requiring CDC via logical replication from the PostgreSQL source of truth [39]. For ROCA’s current scale as a religious organization platform, PostgreSQL with GIN indexes and Redis caching should be sufficient, with OpenSearch evaluation deferred to the medium-priority remediation phase.

CloudFront API Caching. Event metadata API responses (GET /events/{id}) for published events are largely static and can be cached at CloudFront with 60–300 second TTLs to offload the origin entirely for popular events. This reduces Lambda invocations and RDS query load for the most frequently accessed event detail pages.

8 Pillar 5: Cost Optimization and Sustainability

PostgreSQL-First Search Strategy. Maintaining PostgreSQL as the primary search engine avoids the operational and financial overhead of a separate search service. OpenSearch Serverless charges for compute (OCUs) and storage separately [40], and the minimum deployment cost can be significant for a community-focused application. By investing in GIN index optimization and Redis cache-aside patterns, ROCA can defer OpenSearch adoption until query complexity or data volume genuinely demands it.

Infrastructure Right-Sizing. RDS Proxy adds approximately \$0.015 per vCPU-hour but prevents connection exhaustion that would otherwise require scaling to larger RDS instances. Step Functions Standard Workflows charge per state transition (approximately \$0.025 per 1,000 transitions), which is cost-effective for cancellation sagas that execute infrequently compared to the alternative of custom orchestration code with its associated maintenance burden. SQS FIFO queues charge per million requests with no minimum, making them economical for waitlist promotion volumes typical of community events.

Sustainability Considerations. The primary sustainability improvements align with performance and cost optimizations. Redis caching reduces redundant database queries, lowering RDS compute utilization. CloudFront caching for event metadata eliminates unnecessary Lambda cold starts and origin fetches. Materializing recurring event instances on a rolling 90-day window (rather than expanding on every read) reduces per-request compute. Right-sizing Lambda memory allocations for payment-processing functions — which may require more memory for Stripe SDK operations — ensures compute resources match actual workload requirements without overprovisioning.

9 Prioritized Remediation Roadmap

The following roadmap organizes all identified gaps into a 90-day remediation plan across three phases, with effort estimates and cost impact for each item.

Chart Explanation: This Gantt-style chart visualizes the 14 remediation items across three phases. Phase 1 (Days 1–30) addresses six critical items focused on payment security and data integrity. Phase 2 (Days 31–60) tackles four high-priority items for orchestration and parsing reliability. Phase 3 (Days 61–90) covers four medium-priority performance optimizations.

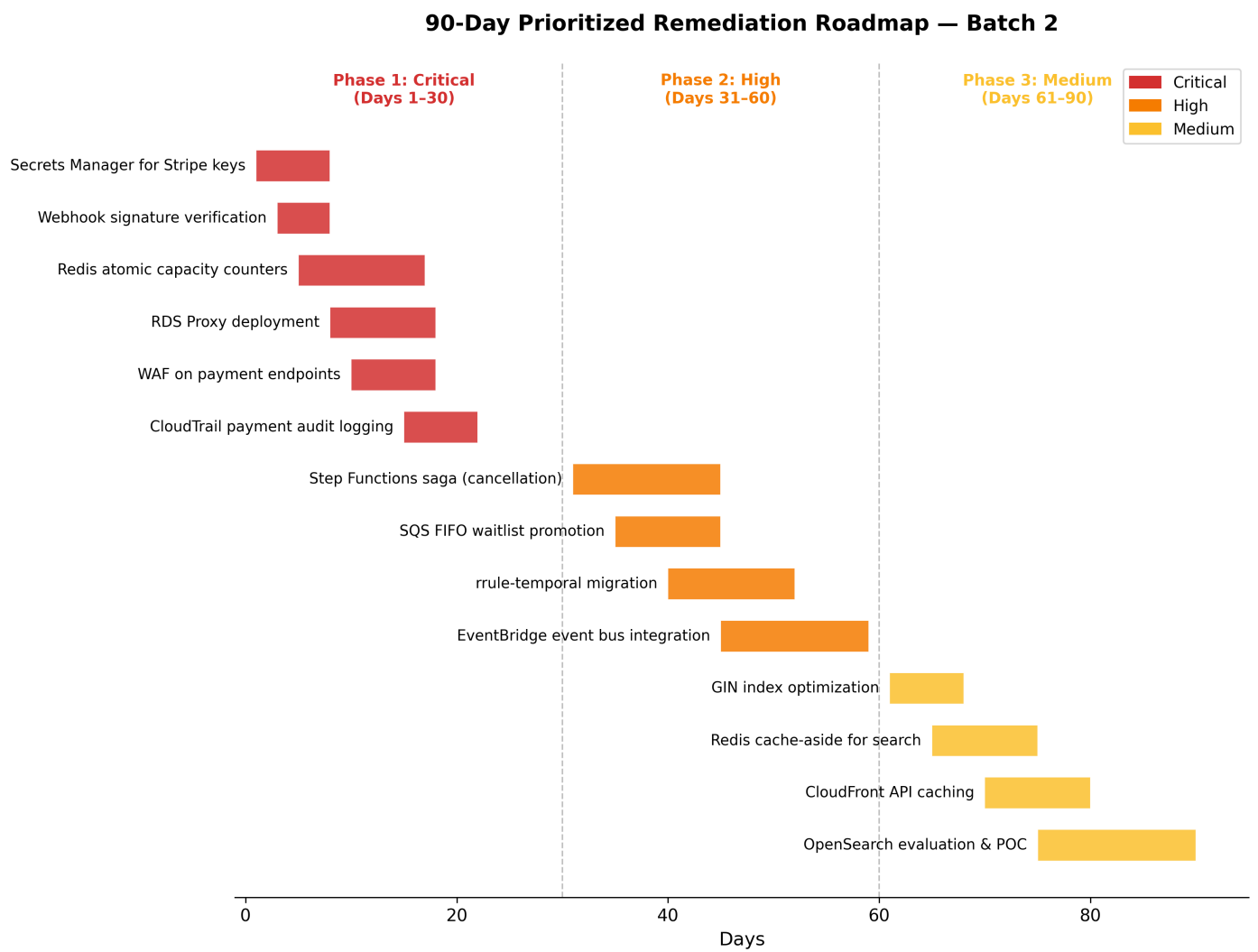


Figure 2: 90-Day Remediation Roadmap

9.1 Phase 1: Critical Remediation (Days 1–30)

Remediation Item	Effort	Monthly Cost Impact	Rationale
Migrate Stripe keys to Secrets Manager [1, 2]	2–3 days	~\$5/secret/month	PCI DSS v4.0 mandatory key management
Implement webhook signature verification [3, 28]	1–2 days	None (code change)	Prevents forged payment events
Deploy Redis atomic capacity counters [7]	3–5 days	Included in ElastiCache	Eliminates RSVP overbooking race conditions
Deploy RDS Proxy [31]	2–3 days	~\$0.015/vCPU-hour	Prevents Lambda connection exhaustion
Attach WAF to API Gateway payment endpoints [4]	2–3 days	~\$5/month + \$0.60/M requests	Rate-limiting and bot protection for payments
Enable CloudTrail payment audit logging [2, 5]	1–2 days	~\$2/100K events	PCI DSS audit trail requirement

Table: Phase 1 Critical Remediation Items with Effort and Cost Estimates

9.2 Phase 2: High-Priority Remediation (Days 31–60)

Remediation Item	Effort	Monthly Cost Impact	Rationale
Step Functions saga for cancellation/refunds [15, 16]	5–7 days	~\$0.025/1K transitions	Compensating transactions for refund failures
SQS FIFO for waitlist promotion [30]	3–4 days	~\$0.50/M requests	Guarantees ordered, exactly-once promotion
Migrate to rrule-temporal [19]	4–5 days	None (library swap)	Fixes DST/timezone defects in recurring events
EventBridge event bus integration [17]	5–7 days	~\$1/M events	Fan-out for cancellation, RSVP, payment events

Table: Phase 2 High-Priority Remediation Items with Effort and Cost Estimates

9.3 Phase 3: Medium-Priority Remediation (Days 61–90)

Remediation Item	Effort	Monthly Cost Impact	Rationale
GIN index optimization on tsvector [33]	2–3 days	None (DB config)	Eliminates sequential scans on search queries
Redis cache-aside for search results [35]	3–4 days	Included in ElastiCache	Reduces DB load for repeated queries
CloudFront API caching for event metadata	2–3 days	~\$0.085/GB transfer	Offloads origin for popular event pages
OpenSearch Serverless evaluation & POC [37, 38]	5–7 days	~\$175/month minimum	Future-proofing for complex search at scale

Table: Phase 3 Medium-Priority Remediation Items with Effort and Cost Estimates

The total estimated effort across all three phases is approximately 40–55 engineering days, with incremental monthly infrastructure costs of roughly \$190–\$250 depending on traffic volume. The critical Phase 1 items should be prioritized immediately given PCI DSS v4.0’s mandatory compliance deadline has already passed [6] and the financial risk exposure from unprotected payment endpoints and RSVP race conditions. Phase 2 items address operational resilience gaps that, while not immediately exploitable, create significant incident response burden when cancellation cascades or refund failures occur. Phase 3 optimizations improve user experience and reduce operational costs but do not represent compliance or data integrity risks.

10 References

1. <https://aws.amazon.com/secrets-manager/faqs>
2. <https://docs.aws.amazon.com/secretsmanager/latest/userguide/monitoring-cloudtrail.html>
3. <https://docs.stripe.com/webhooks/signature?locale=en-GB>
4. <https://docs.aws.amazon.com/securityhub/latest/userguide/pci-standard.html>
5. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/verify-ops-best-practices-pci-dss-4.html>
6. <https://docs.aws.amazon.com/prescriptive-guidance/latest/aws-security-reference-architecture-pci-data-security-standard-dss/introduction.html>
7. <https://redis.io/docs/latest/develop/programmability/eval-intro/>
8. <https://www.cybertec-postgresql.com/en/select-for-update-considered-harmful-postgresql/>
9. <https://github.com/jkbrzt/rrule/issues/550>
10. <https://github.com/jakubroztocil/rrule/issues/253>
11. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=9bf52b11-c4b6-417e-a077-cb09f9214287>
12. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=8ed3eb08-5b4f-4464-bf0e-ad8ba2c4e4b5>
13. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=c7258398-fa22-4df7-8c1c-ecf307ce2a78>
14. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=fbca574c-5c09-48af-a211-8fca0f384548>
15. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
16. <https://aws.amazon.com/jp/blogs/compute/building-a-serverless-distributed-application-using-a-saga-orchestration-pattern/>
17. <https://docs.aws.amazon.com/step-functions/latest/dg/eventbridge-integration.html>
18. <https://github.com/jakubroztocil/rrule/issues/440>
19. <https://github.com/ggaabe/rrule-temporal>
20. <https://github.com/jens-maus/node-ical/>
21. <https://cli.nylas.com/guides/recurring-calendar-events-api>
22. <https://www.sqlservercentral.com/forums/topic/data-storage-strategies-a-balancing-act>
23. <https://docs.aws.amazon.com/scheduler/latest/UserGuide/what-is-scheduler.html>
24. <https://aws.amazon.com/tutorials/scheduling-a-serverless-workflow-step-functions-amazon-eventbridge-scheduler/>
25. <https://stripe.com/resources/more/pci-tokenization>
26. <https://docs.stripe.com/security/guide>
27. <https://www.pcicompliance.com/pci-serverless/>
28. <https://hookdeck.com/webhooks/platforms/guide-to-stripe-webhooks-features-and-best-practices>

29. <https://www.netdata.cloud/academy/update-skip-locked/>
30. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-fifo-queues.html>
31. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html>
32. <https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>
33. <https://www.prisma.io/blog/you-dont-need-elasticsearch-postgres-already-has-full-text-search>
34. <https://xata.io/blog/postgres-full-text-search-engine>
35. <https://redis.io/tutorials/howtos/solutions/microservices/caching/>
36. <https://redis.io/docs/latest/develop/clients/client-side-caching/>
37. <https://aws.amazon.com/blogs/big-data/implement-multi-tenant-search-with-amazon-opensearch-serverless-next-generation/>
38. <https://aws.amazon.com/cn/blogs/big-data/amazon-opensearch-serverless-cost-effective-search-capabilities-at-any-scale/>
39. <https://xata.io/blog/postgres-full-text-search-postgres-vs-elasticsearch>
40. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/cost-optimization.html>